

The Standard of Data Structure – Basic

자료구조의 정석 기초

04 스택과 큐

목차 04 스택과 큐

1. 스택

2. 큐

1. 스택

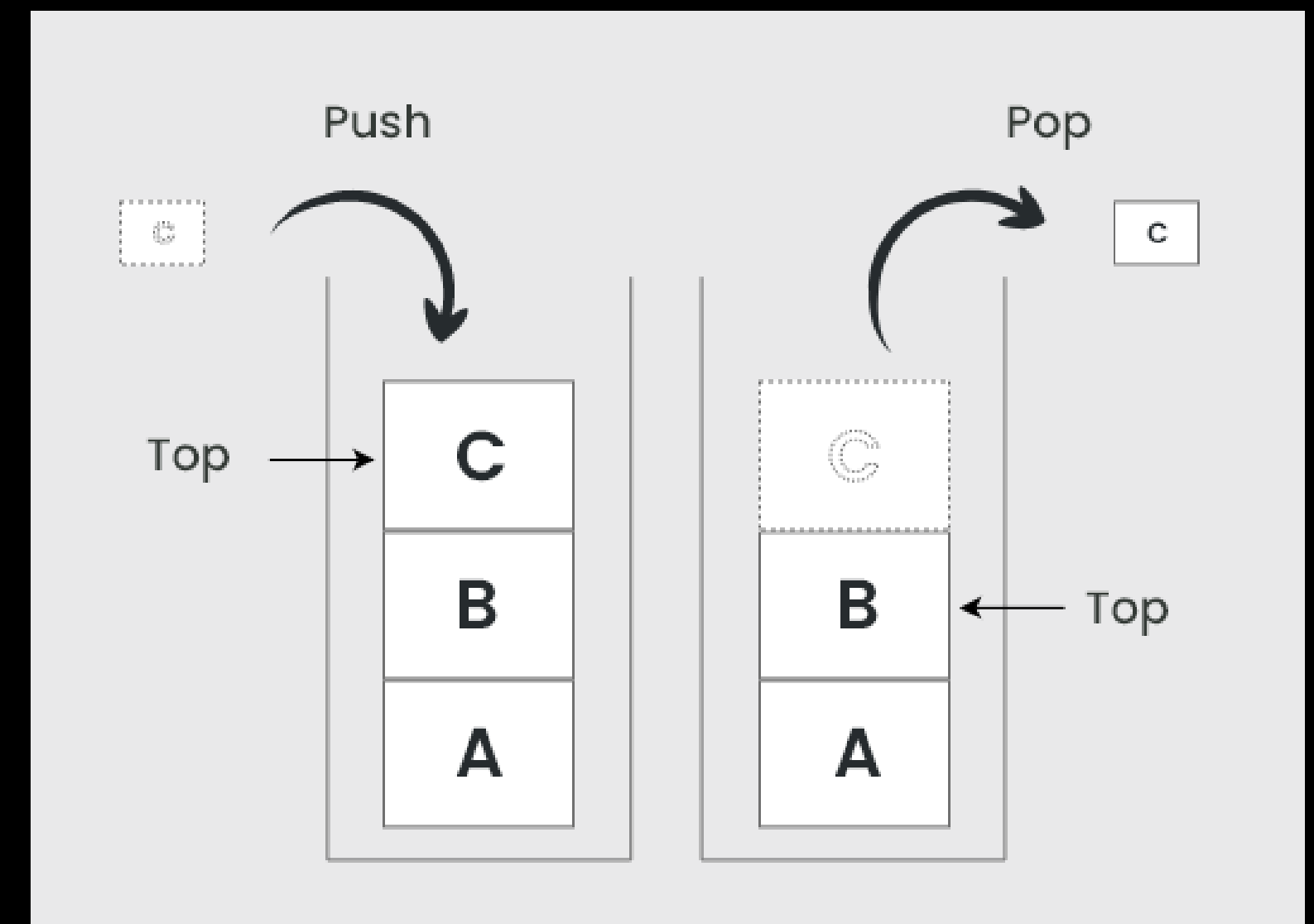


스택(Stack) : 쌓아놓은 더미



• 후입선출(LIFO : Last In First Out)

- First In Last Out
- 가장 최근에 들어온 데이터가 가장 먼저 나감
- 이로 인해 출력 순서가 입력 순서의 역순일 때 자주 사용하는 자료구조



• Push

- 데이터를 스택의 가장 위에 추가

• Pop

- 스택의 가장 위에 있는 요소(element)를 제거

• Top

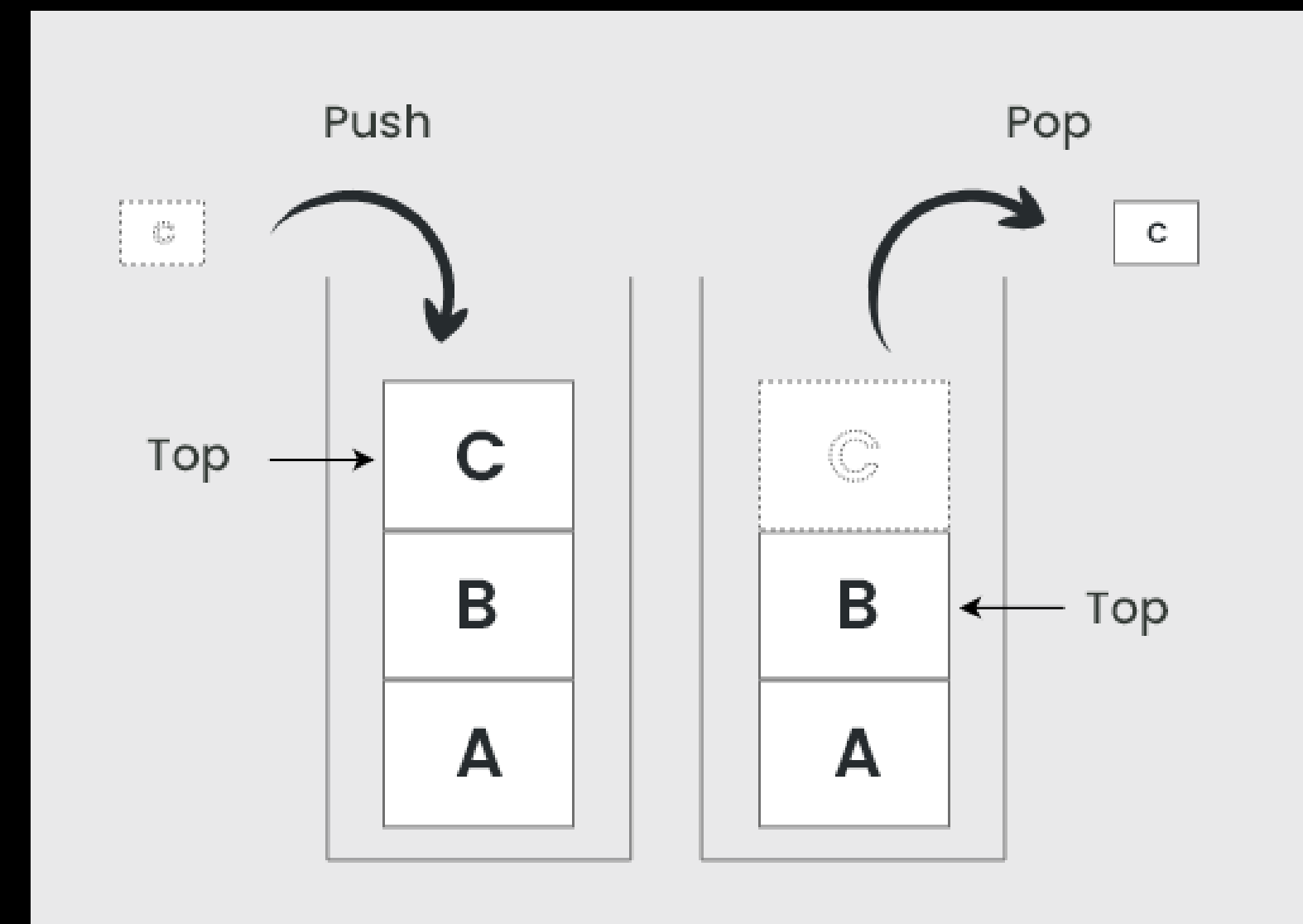
- 스택의 가장 위에 있는 요소를 반환

• Empty

- 스택이 비어 있는지 확인

• Size

- 스택에 있는 요소의 개수를 반환





- **C++**
 - std::stack 클래스 활용
- **Python**
 - Deque를 활용하여 쉽게 구현 가능
- **Java**
 - Java.util.Stack 클래스 사용

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
    stack<int> stack;
}
```

C++

```
from collections import deque

stack = deque()
```

Python

```
import java.util.Stack;

public class StackExample {
    public static void main(String[] args) {
        Stack<Integer> stack = new Stack<>();
    }
}
```

Java



• Push

- 데이터를 스택의 가장 위에 추가

```
stack.push(1); // stack의 맨 위에 1 추가  
stack.push(2); // stack의 맨 위에 2 추가
```

C++, Java

```
stack.append(1) # stack의 맨 위에 1 추가  
stack.append(2) # stack의 맨 위에 2 추가
```

Python



• Pop

- 스택의 가장 위에 있는 요소(element)를 제거

• Top

- 스택의 가장 위에 있는 요소를 반환
- 언어마다 사용하는 메소드가 다름

```
stack.top() // 스택의 맨 위에 있는 요소(2) 반환  
stack.pop() // 스택의 맨 위에 있는 요소(2) 제거
```

C++

```
stack[-1] # 스택의 맨 위에 있는 요소(2) 반환  
stack.pop() # 스택의 맨 위에 있는 요소(2) 제거 및 반환
```

Python

```
stack.peek() // 스택의 맨 위에 있는 요소(2) 반환  
stack.pop() // 스택의 맨 위에 있는 요소(2) 제거 및 반환
```

Java



• Empty

- 스택이 비어 있는지 확인
- 언어마다 사용하는 메소드가 다름

```
if stack :  
    print("스택이 비어 있지 않습니다")  
else :  
    print("스택이 비어 있습니다.")
```

Python

```
if (!stack.empty()) {  
    cout << "스택이 비어 있지 않습니다." << endl;  
} else {  
    cout << "스택이 비어 있습니다." << endl;  
}
```

C++

```
if (!stack.isEmpty()) {  
    System.out.println("스택이 비어 있지 않습니다.");  
} else {  
    System.out.println("스택이 비어 있습니다.");  
}
```

Java



• Size

- 스택에 있는 요소의 개수를 반환

```
cout << stack.size() << endl;
```

C++

```
print(len(stack))
```

Python

```
System.out.println(stack.size());
```

Java

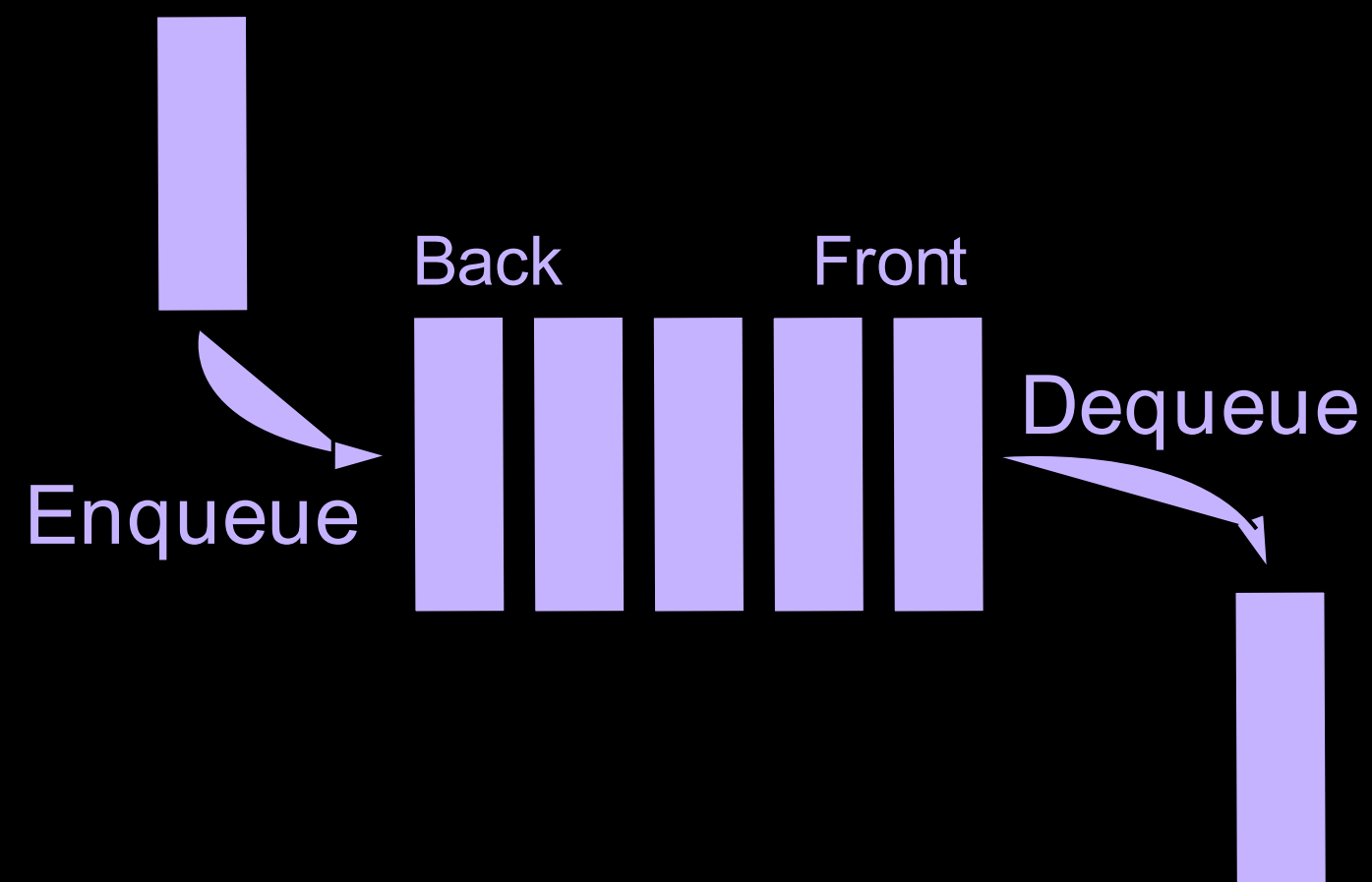
2. 큐



한 줄로 기다리는 대기 줄



- 선입선출(FIFO : First In First Out)
 - Last In Last Out
 - 가장 최근에 들어온 데이터가 가장 나중에 나감
 - 입력된 순서대로 처리해야 할 상황에서 자주 사용하는 자료구조



- **Enqueue**

- 큐의 맨 뒤에 요소 추가

- **Dequeue**

- 큐의 맨 앞 요소를 제거

- **Front**

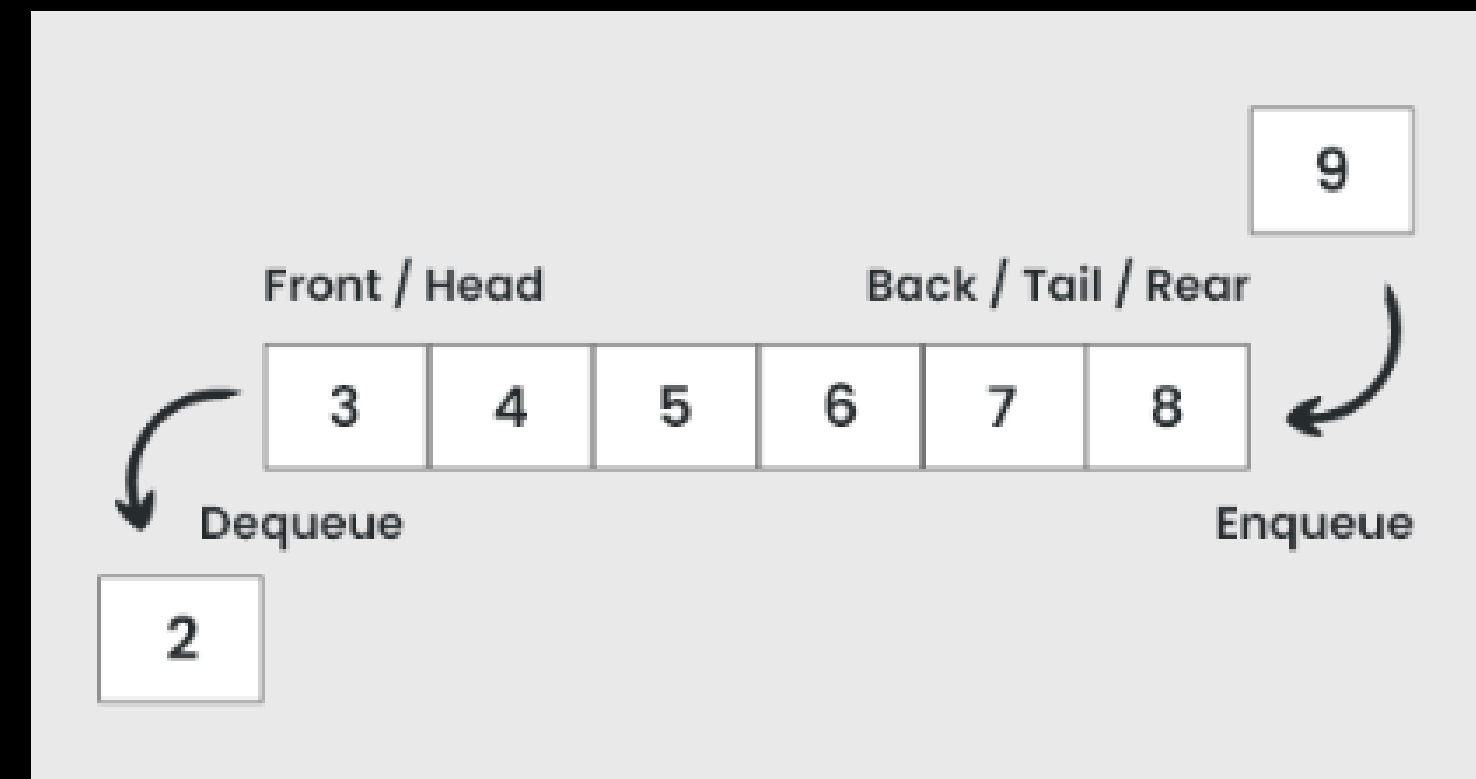
- 큐의 맨 앞 요소를 반환

- **Empty**

- 큐가 비어 있는지 확인

- **Size**

- 큐에 있는 요소의 개수를 반환



- C++
 - std::queue 활용
- Python
 - Deque 활용
 - 스택과 동일
- Java
 - Java.util.Queue 클래스 사용

```
#include <iostream>
#include <queue>

int main() {
    queue<int> queue;
}
```

C++

```
from collections import deque

queue = deque()
```

Python

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueExample {
    public static void main(String[] args) {
        Queue<Integer> queue = new LinkedList<>();
    }
}
```

Java

- Enqueue

- 큐의 맨 뒤에 요소 추가

```
queue.push(1); // 큐의 맨 앞에 1 추가  
queue.push(2); // 큐의 맨 앞에 2 추가
```

C++

```
queue.append(1); # 큐의 맨 앞에 1 추가  
queue.append(2); # 큐의 맨 앞에 2 추가
```

Python

```
queue.add(1); // 큐의 맨 앞에 1 추가  
queue.add(2); // 큐의 맨 앞에 2 추가
```

Java

- Dequeue

- 큐의 맨 앞 요소를 제거

- Front

- 큐의 맨 앞 요소를 반환
 - 언어마다 사용하는 메소드가 다름

```
queue.front() // 큐의 맨 앞에 있는 요소(1) 반환  
queue.pop() // 큐의 맨 앞에 있는 요소(1) 제거
```

C++

```
queue[0] # 큐의 맨 앞에 있는 요소(1) 반환  
queue.popleft() # 큐의 맨 앞에 있는 요소 제거
```

Python

```
queue.peek() // 큐의 맨 앞에 있는 요소(1) 반환  
queue.poll() // 큐의 맨 앞에 있는 요소(1) 제거
```

Java



• Empty

- 큐가 비어 있는지 확인
- 언어마다 사용하는 메소드가 다름

```
if queue :  
    print("큐가 비어 있지 않습니다.")  
else :  
    print("큐가 비어 있습니다.")
```

Python

```
if (!queue.empty()) {  
    cout << "큐가 비어있지 않습니다." << endl;  
} else {  
    cout << "큐가 비어있습니다." << endl;  
}
```

C++

```
if (!queue.isEmpty()) {  
    cout << "큐가 비어있지 않습니다." << endl;  
} else {  
    cout << "큐가 비어있습니다." << endl;  
}
```

Java



• Size

- 큐에 있는 요소의 개수를 반환

```
cout << queue.size() << endl;
```

C++

```
print(len(queue))
```

Python

```
System.out.println(queue.size());
```

Java